Autonomous Software Agents: Deliveroo.js

Davide Donà – davide.dona-1@studenti.unitn.it
264467
Andrea Blushi – andrea.blushi@studenti.unitn.it
264468

University of Trento

June 2026

Contents

Our Solution

- **BDI core** — sense → deliberate → plan → act; collects, explores, navigates
- **PDDL planner** — pushes crates when A* is blocked
- **LLM layer** — interprets mission commands; injects goals, rules, penalties
- **Coordination** — 2PC protocols, peer-injection, team coordinator

Beliefs: Tracker

- Stores latest observation $\langle v, t_{seen} \rangle$ per entity
- Updated on each sensing report
- Invalidated when entity is visible-but-absent

Confidence decay

$$c(t) = \exp\left(-\frac{t - t_{seen}}{\tau}\right)$$

Beliefs: Memory

- Bounded temporal history of observations per entity
- Entries older than a configurable TTL are evicted automatically
- Tracks enemy position history for heat-field computation

Beliefs: Agents

- **Self** — own position and current state
- **Friends** — latest state per teammate via Tracker
- **Enemies** — latest state via Tracker; position history via Memory
- **EnemyPredictor** — estimates each enemy's next position from history
- Spatial heat field — aggregates past enemy observations across the map

Beliefs: Map & Crates

- Static tile layout, transient blocked tiles, traversal penalties
- Crates tracked as dynamic obstacles; latest position via Tracker;

Beliefs: Map — Safe Zones

Directional Tiles create one-way edges — may trap agents in dead zones

Safe SCC (Tarjan's algorithm)

- Map is represented as a directed graph with edges for valid movements;
- Tarjan's algorithm identifies SCCs in this graph;
- An SCC is *safe* iff it contains ≥ 1 spawn tile **and** ≥ 1 delivery tile;
- All tiles outside safe SCCs are reclassified as walls.

Beliefs: Parcels

- Reward decay is applied even when out of sensor range
- Parcel p removed from beliefs when $r_p(t) \leq 0$

Estimated reward

$$r_p(t) = r_{p,0} - \left\lfloor \frac{t - t_0}{\Delta_t} \right\rfloor$$

Desire: Distance Metric

- A* search with dynamic tile costs based on beliefs
- Walls and blocked tiles are impassable; crates are high-cost
- Distance d is the total cost of the A* path to the target

Desire: ReachParcel

- Navigate to and collect a visible parcel
- Priority 1 — competes directly with DeliverParcel

Score

$$S = \frac{r_p + \sum_{c \in C} r_c}{d} \cdot \gamma, \qquad \gamma = \min\left(1, \frac{d_{enemy}}{d}\right)$$

- r_p: parcel reward;
- $\sum_{c \in C} r_c$: value of carried stack;
- γ: race factor — penalizes parcels where enemies have advantage;

Desire: DeliverParcel

- Navigate to delivery tile while carrying parcels
- Priority 1 — competes directly with ReachParcel

Score

$$S = \frac{\sum_{c \in C} r_c}{d}$$

Desire: Explore

- Seek unvisited spawn tile when no parcels are reachable
- Priority 0 — fallback;
- w: cluster density; τ: freshness; h: enemy heat

Score

$$S = \frac{w \cdot \tau}{d \cdot (1 + h)}$$

Desire: Explore — Variables

Freshness τ

$$\tau = \min\left(1,\ \frac{\Delta t}{T_{\text{spawn}}}\right)$$

Δt: time since spawn tile last observed; T_{spawn}: spawn interval

Cluster weight w

$$w = \sum_{y \in OST} \frac{1}{\delta(x,\ y) + 1}$$

OST: spawn tiles within observation range R_{obs}

Enemy heat h

$$h = \sum_{e \in E} \exp\left(-\frac{d_e^2}{2\sigma^2}\right) \exp\left(-\frac{\Delta t_e}{\tau_h}\right)$$

Spatial $\times$ temporal decay over all tracked enemy observations

Collision: Detection

Checked before every move step. Target tile is *occupied* if:

- An enemy is currently observed on it, **or**
- An enemy's *predicted* next position matches it with sufficient confidence

Enemy position prediction

Next position estimated from last known state and movement history; only predictions above a confidence threshold trigger a block.

Collision: Escalation

1. **Detour** — replan with tile as wall; accepted only if extra steps $\leq \Delta_{th}$
2. **Wait** — no viable detour: pause for random duration $\sim \mathcal{U}$
3. **Block & replan** — after wait expires *or* after L repeated invalidations: mark tile blocked for random TTL, full replan

[collision resolution]

When A* Fails

- A* fails: no path that avoids crates
- Second A* ignoring crates finds a “ghost” path
- Crates intersecting that path are part of the “PDDL“ problem instance

PDDL

- Cluster detected → PDDL problem built from current beliefs
- Solver finds `move` + `push` sequence to reach goal
- Result translated back to agent's action plan

The PDDL Domain

- STRIPS: two object types — `tile` and `crate`
- 4 directional **move** actions (step into crate-free tile)
- 4 directional **push** actions:
  - Agent adjacent on opposite side of push direction
  - Destination must be a valid, crate-free tile
- Push relocates crate; agent steps onto its former tile

Performance Bottlenecks

- **Real-Time Constraints:** The initial PDDL solver integration was too slow to meet the game's strict timing demands.
- **Identified Issues & Fixes:**
  - *API Overhead:* Remote solver calls introduced heavy latency. **Fix:** Migrated to a local solver instance.
  - *Redundant Computation:* Identical crate configurations were being solved repeatedly, but implementing a standard cache was non-trivial.
- **The Pivot:** Rather than brute-forcing a complex caching mechanism, we restated the problem to naturally maximize cache hits.

Optimization: Problem Restatement

Three-Phase Journey Segmentation

1. **Approach:** A* pathfinding from the current position to the cluster *entry*.
2. **Manipulation:** PDDL execution solely through the crate cluster (entry $\rightarrow$ exit).
3. **Completion:** A* pathfinding from the cluster *exit* to the final goal.

Optimization: Problem Restatement

- **The Result:** The PDDL problem is now strictly anchored to the cluster entry tile rather than absolute map coordinates.
- **Simplified Cache Key:** $\langle$entry, exit, $\{$crate positions$\}\rangle$.

[segmentation result video]

LLM Layer Overview

- Thin wrapper around the BDI agent
- Listens for mission-controller chat messages
- Up to 4 tool-call rounds (hops) per message
- Three command categories: **goals** · **rules** · **coordination**

Goal Injection

- **go-to** $\rightarrow$ `REACH_TILE` desire (configurable reward + TTL)
- **deliver-at** $\rightarrow$ `DELIVER_PARCEL` at specific tile
- Both compete with parcel goals via standard scoring
- Persist and re-evaluated until executed or TTL expires

Rule Injection

- `RuleStore`: condition + multiplier m + offset a
- Applied each cycle: scales/shifts baseline desire scores
- Example: incentivise collecting n parcels before delivery
- Stack rule automatically elevates Explore to priority 1

Traversal Penalties

- Adds additive cost to A* edge leading onto a tile
- Agent prefers detour if detour $<$ penalty length
- Tile not forbidden — used as last resort
- Operates at the planning layer, not the deliberation layer

Team Communication

- **Beacons** — each agent broadcasts position at fixed interval
- Ensures up-to-date peer positions beyond sensing radius
- **Peer-injection** — command applied locally, then broadcast
- Simple commands (goals, rules, penalties) are fire-and-forget

Holds: 2-Phase Commit

`request_rendezvous` & `request_red_light` use 2PC:

- **Phase 1** — propose broadcast; each teammate votes
- Vote: accept if hold-tile score beats current best goal
- **Phase 2** — unanimous accept → commit; else abort
- Red light: hold on nearest odd-row tile; green light clears all holds

[traffic control]

Coordinator

- Periodic rounds: broadcast `request_beliefs` to team
- Collects positions, payloads, high-value zones
- LLM receives geometric summary $\rightarrow$ issues `assign_goto` commands
- Distributes agents across the map optimally
- Also triggered on-demand via `request_team_status`